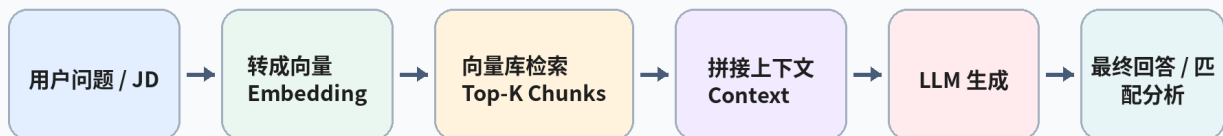


RAG 原理与完整流程

以 AI 招聘助手 / 简历匹配项目为例

Retrieval-Augmented Generation (检索增强生成)

RAG 一句话流程：先找资料，再带着资料让大模型回答



核心点：RAG 不要求模型“背住”你的资料，而是在回答前临时检索最相关的原文片段。

三句话讲清楚 RAG

RAG = Retrieval (检索) + Augmented (增强) + Generation (生成)。当用户提问时，系统先从自己的知识库里找出最相关的原文片段，再把这些片段连同问题一起交给大模型。这样模型不是只靠训练时记住的知识“猜答案”，而是基于刚检索到的真实资料来回答。

适合用途：项目理解 · 面试复习 · RAG 调试 · 项目答辩说明

1. 先把 RAG 真正理解成一个“开卷考试”系统

最容易理解的类比：普通大模型像“闭卷考试”，只能依赖自己训练时学过的知识；RAG 像“开卷考试”，回答前可以先去翻你提供的资料。

一句话公式

用户问题 -> 检索相关资料 -> 把资料加入上下文 -> LLM 根据问题 + 资料生成回答

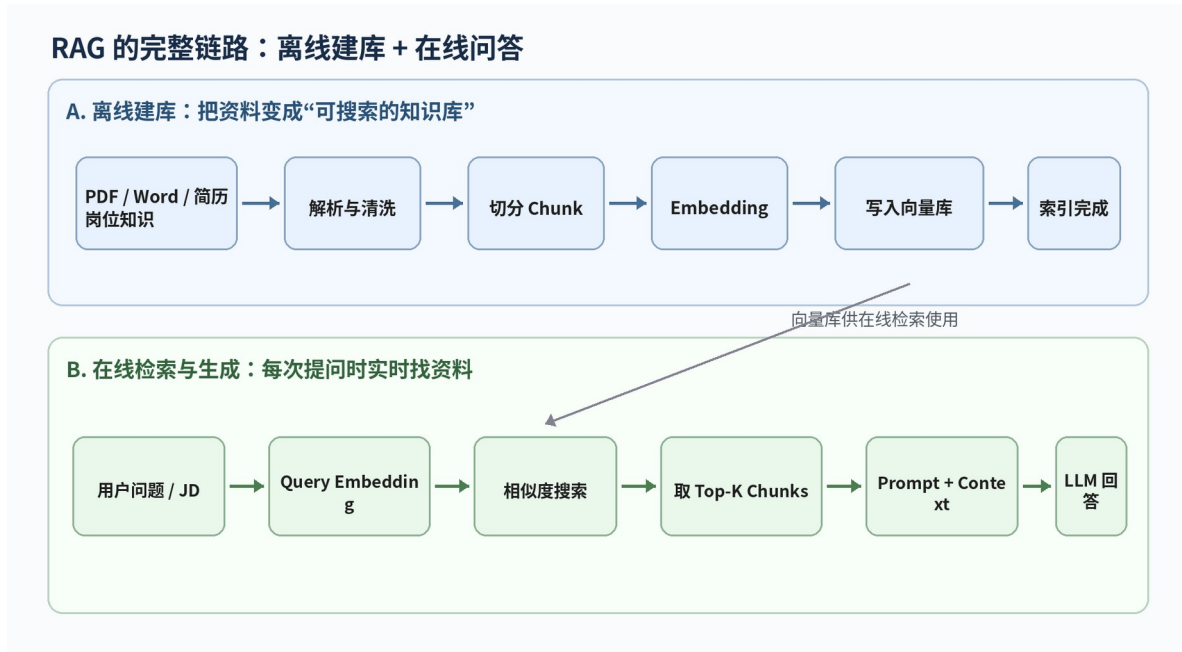
为什么项目里需要 RAG ？

- 大模型本身不知道你刚刚上传的简历、JD、企业内部文档或最新项目资料。
- 把所有文档一次性全部塞进 Prompt，成本高、上下文长，而且容易把真正相关的信息淹没。
- RAG 只挑“当前问题最相关”的几个 Chunk，所以更省 Token，也更容易给出可解释证据。
- 知识更新时通常只需要更新知识库，不需要重新训练大模型。

RAG 不等于“训练模型”

方式	核心动作	资料如何进入答案
直接问 LLM	不额外提供资料	主要依赖模型训练知识
Fine-tuning 微调	改变模型参数/行为	知识可能进入参数，但更新成本高
RAG	先检索，再生成	每次问答动态注入相关原文

2. RAG 的完整流程：分成两个阶段



记忆口诀

先建库，再检索；先找证据，再让模型说。

A. 离线建库阶段：只做一次或资料更新时重做

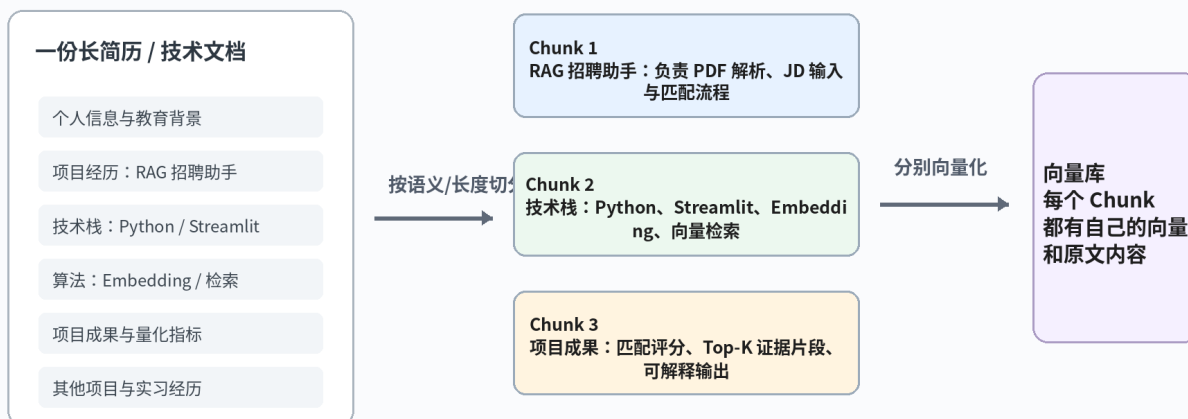
- 1) **读取资料**：例如 PDF 简历、Word 文档、岗位说明、项目材料。
- 2) **解析与清洗**：提取可用文本，去掉无关空白、页眉页脚、乱码等。
- 3) **切分 Chunk**：把长文本切成多个小片段。每个 Chunk 最好语义相对完整，避免过长。
- 4) **Embedding 向量化**：使用 Embedding 模型把每个 Chunk 转成一串数字向量，用来表达语义。
- 5) **写入向量数据库**：保存“向量 + Chunk 原文 + 来源/页码等 metadata”，并建立可检索索引。

B. 在线问答阶段：每次用户提问都会执行

- 1) **接收 Query**：例如用户输入一段 JD，或者问“候选人是否满足 RAG 岗位要求？”
- 2) **Query Embedding**：把用户问题也转换成与知识库相同维度的向量。
- 3) **相似度搜索**：比较 Query 向量与各 Chunk 向量的语义距离。
- 4) **取 Top-K**：拿到相似度最高的 K 个片段，例如 Top-3、Top-5。
- 5) **组装 Prompt**：把问题 + 检索到的 Chunk + 回答规则拼在一起。
- 6) **LLM 生成**：大模型基于这些证据完成总结、匹配分析、评分或回答。

3. Chunk：RAG 里最关键但最容易被忽略的一步

Chunk 是什么？为什么必须切分？



好处：问题只需要匹配最相关的小片段，而不是把整份文档全部塞给模型。

Chunk 就是一小段文本。如果一份简历有 3000 字，系统不会把整份简历看成一个巨大向量，而是通常切成多个片段，每个片段单独 Embedding。

为什么 Chunk 不能太大？

- 一个大 Chunk 可能同时包含很多主题，向量语义会被“平均”，导致检索不够精准。
- 检索到大 Chunk 后会把大量无关文字带进 Prompt，浪费上下文和 Token。
- 大模型看到太多无关信息时，反而更难聚焦关键证据。

为什么 Chunk 也不能太小？

- 太短会丢失上下文，例如只剩“负责模型优化”却不知道优化什么模型。
- 一个完整事实可能被切断，导致检索片段缺少足够证据。

常见工程参数（不是固定答案）

可以从 chunk_size 约 300-800 个中文字符、chunk_overlap 约 50-150 个字符开始试，再根据召回效果调整。真正标准不是某个固定数字，而是：检索出来的片段是否“完整、相关、足够支撑回答”。

4. Embedding 和相似度：系统到底是怎么“找到相关 Chunk”的？

Embedding 的作用：把一段文字压缩成一个高维数字向量。语义越接近的两段文字，向量通常越接近。

对象	文本
Query	“需要熟悉 RAG、向量数据库、Python 的 AI 工程实习生”
Chunk A	“项目使用 Python + Streamlit，实现 Embedding 与 Top-K 向量检索”
Chunk B	“熟悉 MySQL，参与后端接口开发”
Chunk C	“获得校园篮球比赛二等奖”

直觉上，Chunk A 与 Query 的语义最接近，所以它的相似度分数应该最高，会优先被检索出来。

常见相似度

余弦相似度常写成： $\text{similarity} = (q \cdot d) / (\|q\| \times \|d\|)$ 。你不需要先死记公式，只要记住：分数越高，一般表示两个向量方向越接近、语义越相关。

Top-K 是什？

Top-K = 取相似度最高的 K 个 Chunk。例如 K=3，就只拿最相关的 3 个片段进入 Prompt。K 太小可能漏证据，K 太大又可能混入噪声，因此需要结合项目调试。

候选 Chunk	示意相似度	Top-K 结果
Chunk A: RAG / Python / 向量检索	0.86	保留
Chunk B: 后端接口 / MySQL	0.58	可能不保留
Chunk C: 篮球比赛	0.12	不保留

5. 检索完成之后：Context 如何进入 Prompt？

检索本身不会生成答案。RAG 的关键是把检索到的原文作为 Context，和用户问题一起交给 LLM。

一个简化的 RAG Prompt

System: 你是一个 AI 招聘分析助手，只能基于提供的证据进行判断。

Context:

[Chunk 1] 候选人使用 Python 和 Streamlit 完成招聘助手前端。

[Chunk 2] 项目包含 Embedding、向量检索、Top-K 证据召回。

[Chunk 3] 候选人实现 PDF 简历解析与结构化处理。

Question:

该候选人是否满足“熟悉 Python、RAG、文档解析”的岗位要求？

Instruction:

逐项对照 JD，并引用相关 Chunk 作为证据；证据不足的地方明确说明。

为什么这一步能降低“胡说”？

因为模型看到的不只是问题，还看到了你系统检索出的证据。工程上还可以进一步要求“仅根据 Context 回答”“引用 Chunk 编号”“证据不足时说不知道”，让输出更可控、更可解释。

6. 放到你的 AI 招聘助手里，可以这样理解

把当前项目抽象成 RAG 后，最重要的是区分“谁是 Query、谁是知识库、谁是最终任务”。

RAG 概念	在招聘助手中的对应	作用
Knowledge Base	解析后的简历内容 / 项目经历 / 技能片段	被切成 Chunk 并向量化
Query	JD 要求，或对某项岗位能力的提问	决定当前要找什么证据
Retriever	Embedding + 向量相似度检索	从简历中找最相关经历
Top-K Chunks	与 JD 最相关的技能/项目片段	作为匹配判断的证据
LLM	生成匹配分析、差距说明、总结	把证据组织成人能读懂的结果
Metadata	Chunk 来源、页码、类别等	帮助解释“证据来自哪里”

举一个你现在最容易遇到的检索例子

JD：“熟悉 RAG、Embedding、向量数据库，具备 Python 开发能力。”

Chunk A：使用 Python + Streamlit 开发 AI 招聘助手。

Chunk B：将简历文本切分后做 Embedding，并实现 Top-K 相似度检索。

Chunk C：负责校园活动组织与宣传。

为什么会检索出 A 和 B？

因为 JD 中“RAG / Embedding / 向量数据库 / Python”的语义，与 A、B 的内容高度接近。即使它们没有逐字完全相同，Embedding 也能根据语义找到相关片段。这就是“语义检索”与传统关键词搜索最大的区别之一。

7. 当 RAG 检索结果不理想时，应该按什么顺序排查？

- ① 先看 **Chunk** 相关事实是不是被切碎了？Chunk 是否过大或过小？
- ② 再看 **Query** JD 是否太长、混合太多要求？是否需要拆成“技能/学历/项目经验”等子问题？
- ③ 看 **Embedding 模型** 中文语义效果是否可靠？建库和 Query 是否使用同一个 Embedding 模型？
- ④ 看 **Top-K** K 太小会漏，K 太大容易引入无关内容。
- ⑤ 看 **相似度阈值** 如果设置阈值，是否过高导致没有结果，或过低导致噪声太多？
- ⑥ 看 **Prompt** 即使检索对了，Prompt 没有限制模型使用证据，也可能生成得很飘。
- ⑦ 看 **最终评估** 不要只看最终回答，要把“Query -> Top-K Chunk -> Prompt -> Answer”整条链一起检查。

调 RAG 的核心思想

先判断“有没有把对的证据找出来”，再判断“模型有没有把证据用对”。前者是 Retrieval 问题，后者是 Generation / Prompt 问题。不要一上来只调大模型。

建议你调试时固定打印/展示的 5 个东西

- 原始 Query
- 每个被召回 Chunk 的文本
- 每个 Chunk 的相似度分数
- 最终拼接给 LLM 的 Context / Prompt
- LLM 最终输出

8. 最后用 30 秒把 RAG 讲给面试官

30 秒版本

RAG 的核心是“检索增强生成”。我会先把文档解析、切分成 Chunk，并通过 Embedding 转成向量写入向量数据库。用户提问时，我把 Query 同样向量化，做语义相似度检索，取 Top-K 相关片段作为 Context，再与问题一起交给 LLM 生成答案。这样能够让大模型基于项目自己的实时资料回答，同时减少幻觉，并且可以展示检索证据提升可解释性。

你现在只需要牢牢记住这 6 个词

Chunk	Embedding	Vector DB
Similarity Search	Top-K	Context + LLM

理解顺序：文档 -> Chunk -> Embedding -> 向量库 -> Top-K 检索 -> Context -> LLM

当你能够解释清楚“为什么这个 JD 会检索出这几个 Chunk”，你就真正开始理解 RAG 了。